

NerdVault

Spiegazione del codice e dell'architettura - guida per l'esame orale

Marketplace PHP 8.2 / MVC a livelli (Entity-Foundation) / Smarty / MySQL-PDO

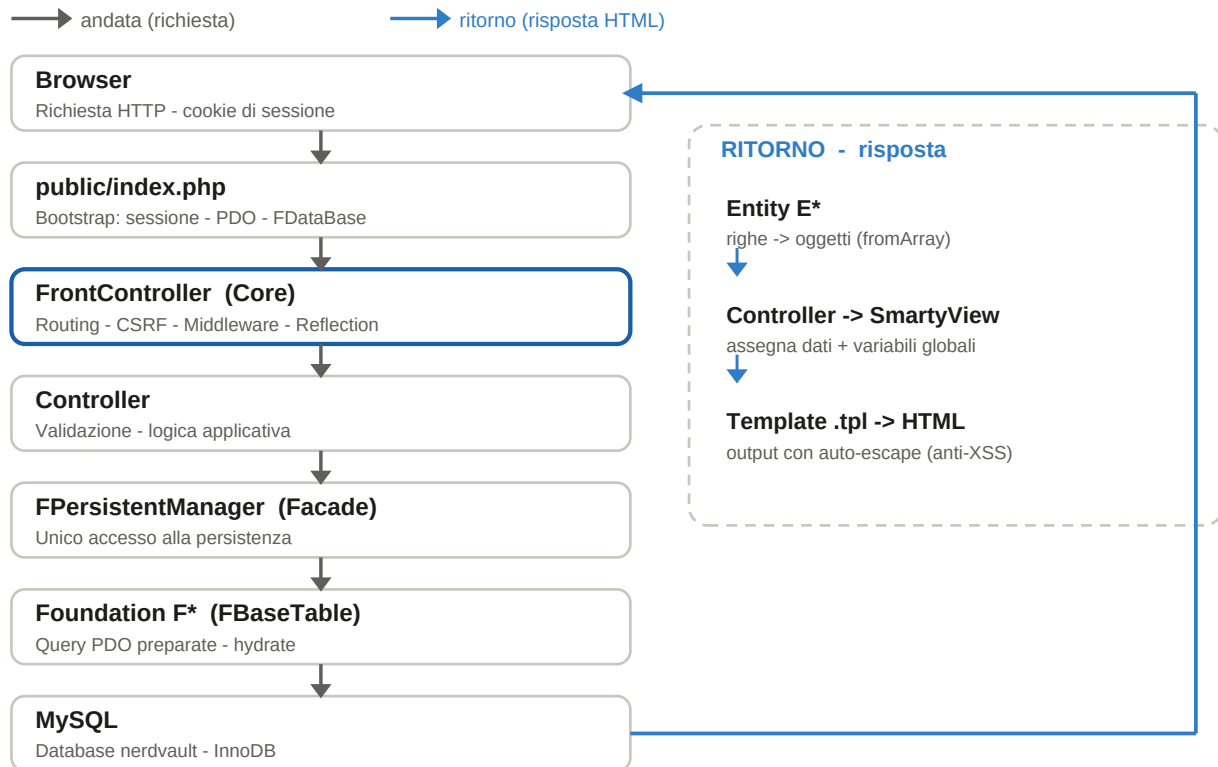


Figura 1 - Ciclo di vita di una richiesta attraverso i livelli

1. Visione d'insieme

NerdVault è un marketplace per oggetti nerd (manga, figure, carte) scritto in **PHP 8.2** con architettura **MVC a livelli**, motore di template **Smarty**, database **MySQL/InnoDB** via **PDO** e dipendenze gestite con **Composer** (autoload PSR-4).

L'architettura segue il pattern **Entity-Foundation**:

- **Entity** (classi E*) = oggetti di dominio in memoria;
- **Foundation** (classi F*) = livello di persistenza (accesso al DB).

Il principio chiave è la **separazione delle responsabilità**: ogni livello conosce solo quello immediatamente sotto. Un Controller non scrive mai SQL; una Foundation non sa nulla di HTTP; una View non accede al database.

2. Il ciclo di vita di una richiesta

È la domanda più frequente all'orale. La richiesta attraversa i livelli così (vedi Figura 1):

- **public/index.php** - unico punto di ingresso (l'.htaccess riscrive ogni URL su index.php).
Bootstrap: carica l'autoloader, avvia la sessione, crea il \$pdo, inizializza il singleton FDataBase::init(\$pdo) e lancia FrontController.

- **FrontController** fa 4 cose in ordine: **routing** (URL -> rotta, match a prefisso più lungo), **controllo del metodo HTTP**, **verifica CSRF** sulle POST, **middleware** di autorizzazione; poi invoca controller + action via **Reflection**.
- Il **Controller** valida l'input, applica la logica e chiama la Facade **FPersistentManager**.
- **FPersistentManager** delega alla **Foundation** giusta, che esegue query **PDO** **preparate** e *idrata* le righe in oggetti **Entity**.
- Gli oggetti tornano al Controller, che usa **SmartyView** per renderizzare un template .tpl in **HTML**.

3. I livelli nel dettaglio

Core (src/Core) - infrastruttura HTTP

- **FrontController**: il dispatcher. La tabella `routes()` mappa 'controllore/azione' su controller, action, metodi, middleware e parametri. Usa `ReflectionMethod` per chiamare solo metodi pubblici e non ereditati da `BaseController` (evita l'invocazione di metodi arbitrari).
- **Request**: incapsula `$_GET/$_POST/$_SERVER/$_FILES`. Nessun altro file legge le superglobali direttamente.
- **SessionManager**: cookie `HttpOnly/SameSite/Secure`, `use_strict_mode`, timeout di inattività (30 min) e **rigenerazione dell'ID al login** (anti session fixation).
- **Csrf**: token casuale in sessione, validato con `hash_equals` (anti timing-attack).

Controller (src/controllers)

Estendono **BaseController** (pattern Template Method) che offre helper comuni: `view()`, `renderError()`, validazioni e il logging delle azioni admin. Ogni controller gestisce un'area (Annuncio, Utente, Carrello, Pagamento, Feedback, Segnalazione, Admin, Business, Wishlist, Home, Legal). Pattern ricorrente: un metodo `xFormOrSubmit` che mostra il form su GET e lo processa su POST.

Foundation (src/Foundation)

- **FDataBase**: Singleton che incapsula la connessione PDO e offre `transaction()` per blocchi atomici con rollback automatico.
- **FBaseTable** (astratta, Template Method): CRUD generico (`find`, `insert`, `updateById`, `deleteById`). Le sottoclassi dichiarano solo `tableName`, `primaryKey`, `entityClass` e `columns (whitelist)`. `column()` valida gli identificatori SQL con regex (difesa injection su nomi tabella/colonna).
- **Classi F* concrete**: query specifiche (es. `FAnnuncio::search`, `findWithDetails` con JOIN).
- **FPersistentManager** (Facade): metodi statici che i controller chiamano; nasconde quale Foundation usare.

Entity (src/Entity)

- **EBaseEntity** (astratta): legge chiavi snake/camelCase, conversioni tipizzate, e il meccanismo extra che conserva i campi dei JOIN nel round-trip oggetto/DB.
- **Classi E***: `fromArray/toArray`, `getter/setter` e **metodi di dominio** con le regole di business (`isAttivo`, `isAcquistabileDa`, `completa`). La logica di dominio vive qui, non sparsa nei controller.

View (src/View + templates) e Middleware

- **SmartyView**: configura Smarty, registra i modifier e assegna le variabili globali (sessione, token CSRF, categorie, contatore carrello). `setEscapeHtml(true)` attiva l'auto-escaping anti-XSS.

- **Middleware:** funzioni di autorizzazione (`requireAuth`, `requireGuest`, `requireAdmin`, `requireAdminLivello2`, `requireBusiness`, `denyAdmin`, `denyBusiness`) associate alle rotte ed eseguite prima dell'action.

4. Pattern di progettazione

Domanda classica: "quali pattern hai usato?"

Pattern	Dove	Perche
Front Controller	FrontController + index.php	un solo ingresso, routing e sicurezza centralizzati
Singleton	FDataBase	una sola connessione PDO condivisa
Facade	FPersistentManager	nasconde il livello Foundation ai controller
Template Method	FBaseTable, BaseController, EBaseEntity	algoritmo generico, dettagli nelle sottoclassi
Factory Method	fromArray(), FDataBase::table(), SmartyView::make()	creazione oggetti centralizzata
Table Data Gateway / Repository	classi F*	un oggetto per tabella che incapsula le query

5. Sicurezza

- **SQL injection**: sempre prepared statement con parametri; identificatori validati da regex.
- **XSS**: auto-escaping globale di Smarty; no filter usato solo dove serve HTML consapevolmente.
- **CSRF**: token in sessione validato su ogni POST.
- **Password**: password_hash (bcrypt) + password_verify; mai in chiaro.
- **Sessione**: cookie HttpOnly/SameSite/Secure, rigenerazione ID al login, timeout di inattività.
- **Autorizzazione**: middleware sui ruoli + controlli di **ownership** nelle query (es. UPDATE ... WHERE id_utente = ?). E' la classe di bug **IDOR** corretta sul feedback: ora si verifica che il pagamento sia dell'utente prima di salvare la recensione.
- **Upload file**: validazione del MIME reale con finfo, limite dimensione, nomi casuali, move_uploaded_file, protezione path traversal in cancellazione.
- **Flussi email**: verifica con token a scadenza, reset password monouso, risposte generiche (anti user enumeration).

6. Modello dei dati e dei ruoli

Tre tipi di attori, volutamente separati:

- **utente_registrato**: utente normale, può comprare e vendere come privato;
- **account_business**: collegato 1:1 a un utente, può solo vendere (denyBusiness blocca carrello/wishlist/acquisto);
- **admin**: tabella separata, con livello_sicurezza 1 o 2 (il 2 modera altri admin); non può comprare (denyAdmin).

Vincoli notevoli: chk_annuncio_venditore impone un solo proprietario (utente o business); pagamento ha UNIQUE(id_annuncio) (un annuncio si vende una volta sola); feedback ha UNIQUE(id_pagamento) (una recensione per acquisto).

7. Esempio end-to-end: acquisto di un annuncio

- L'utente clicca "Acquista" -> GET /pagamento/checkout/5 (middleware: requireAuth, denyAdmin, denyBusiness).
- PagamentoController::checkout verifica che l'annuncio sia attivo e non dell'utente, mostra gli indirizzi.
- POST verso il placeholder PayPal (simulato), poi pagamento/conferma.
- FPersistentManager::confirmSinglePayment gira in **transazione**: carica l'annuncio con FOR UPDATE (lock anti doppia vendita), crea il pagamento, marca l'annuncio venduto solo se ancora attivo (markSoldIfActive, atomico) e lo rimuove da carrelli/wishlist altrui.
- Redirect alla pagina esito.

Questo esempio tocca tutti i livelli e mostra la **gestione della concorrenza** (transazione + lock), molto apprezzata agli esami.

8. Domande tipiche e risposte rapide

Domanda	Risposta
Cos'e la Facade qui?	FPersistentManager: i controller chiamano solo lei, non sanno quale tabella/Foundation c'e dietro.
Come previeni l'SQL injection?	Prepared statement ovunque + whitelist/regex sugli identificatori.
Come gestisci i ruoli?	Middleware associati alle rotte + controlli di ownership nelle query.
Perche Entity e Foundation separate?	L'Entity e l'oggetto di dominio (business); la Foundation e la persistenza. Separarle permette di cambiare DB senza toccare la logica.
Due utenti comprano lo stesso annuncio insieme?	Transazione + SELECT ... FOR UPDATE + update condizionato WHERE stato='attivo': solo uno vince.

Documento generato come supporto allo studio. I riferimenti ai file (es. FrontController.php, FPersistentManager.php) seguono la struttura del repository in src/.